

BlueFolder Rollout Playbook

Presentation and implementation guide for the current BlueFolder-backed OpsHub ecosystem.

Goal

Introduce OpsHub into an existing BlueFolder environment without disrupting daily operations.

The safest model is one OpsHub server-side BlueFolder utility credential and role-specific OpsHub clients.

Recommended Credential Model

Use:

- one BlueFolder utility/API user for OpsHub
- OpsHub operator IDs for RouteDesk and PartsDesk
- FieldDesk technician IDs mapped to BlueFolder technician users
- optional Discord linked identities for notifications and commands

Avoid:

- distributing BlueFolder API credentials to every technician tablet
- compiling BlueFolder credentials into web bundles
- making Discord a hard requirement for non-Discord app users
- letting each frontend write to BlueFolder through its own private implementation

Why One Utility User

Benefits:

- easier credential rotation
- smaller blast radius
- tablets and browsers do not hold BlueFolder API secrets
- one audit/control layer in OpsHub
- simpler Cloudflare/public access model

Tradeoff:

- BlueFolder may show the utility/API user as the technical author of notes or updates.
- OpsHub should include human attribution in note text and keep its own event audit.

Attribution Pattern

Use note/update text like:

```
Submitted via FieldDesk
Operator: tech-mike
BlueFolder technician: 12345
Action: Job note
```

For dispatch:

```
Submitted via RouteDesk
Operator: route-desk
Action: Dispatch disposition
```

For parts:

```
Submitted via PartsDesk
Operator: parts-desk
Action: Part ready
```

Rollout Phases

Phase 1: read-only validation

- connect OpsHub to BlueFolder
- verify health
- verify known SR lookups
- verify assignments
- verify RouteDesk and PartsDesk can load boards
- no live writes yet

Phase 2: controlled writes

- choose a known safe SR
- submit one FieldDesk note
- submit one PartsDesk lifecycle update
- submit one dispatch workflow action if applicable
- inspect BlueFolder output
- decide if attribution is acceptable

Phase 3: tablet pilot

- configure one tablet
- use OpsHub backend mode
- verify Today and Queue order each morning
- capture feedback on photo/closeout behavior
- keep pilot small until confidence is high

Phase 4: desk pilot

- allow one dispatcher to use RouteDesk
- allow one parts operator to use PartsDesk
- log issues in OpsHub, not as ad-hoc BlueFolder workarounds

Phase 5: broader release

- rotate API token if needed
- document operator IDs
- confirm rollback
- train users with module walkthroughs

Rollback Plan

If something looks risky:

- stop the presentation Docker stack
- switch FieldDesk tablet back to direct BlueFolder fallback only if absolutely needed
- keep BlueFolder as the manual operating fallback
- do not rotate or distribute additional BlueFolder API keys in a panic

Release Gate

Do not expand past pilot until:

- OpsHub health is reliable
- RouteDesk and PartsDesk operator IDs are understood
- FieldDesk Today first-stop behavior is correct
- one live note/update attribution pattern has been accepted
- photo/closeout limitations are clearly documented
- the team knows which actions write to BlueFolder and which are read-only

Training Summary

One sentence version:

OpsHub is the controlled bridge between BlueFolder and the role-specific apps; BlueFolder keeps the record, OpsHub owns action, workflow, and safe API access.